

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 11
Operating Systems
Topic: The Race and the Semaphore (Track A - Linux)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Laboratory Objectives 3

2 PART 1: OBSERVING THE DATA RACE **5**

2.1 Program Implementation (`race.c`) 5

2.2 Execution and Trace Output 5

2.3 Answers to Required Observation Questions 5

3 PART 2: SYNCHRONIZATION VIA MUTEX AND SEMAPHORE **7**

3.1 Implementation of `mutex.c` and `sem.c` 7

3.2 Execution and Performance Benchmarking 7

3.3 Answers to Required Observation Questions 8

4 PART 3: A SEMAPHORE AS A SIGNAL **10**

4.1 Program Implementation (`order.c`) 10

4.2 Execution Trace 10

4.3 Answers to Required Observation Questions 10

5 PART 4: COUNTING SEMAPHORES AND RESOURCE LIMITS **12**

5.1 Program Implementation (`parking.c`) 12

5.2 Execution Trace 12

5.3 Answers to Required Observation Questions 13

6 EXTENSION: LOCK-FREE HARDWARE ATOMICS **14**

6.1 Lock-Free Atomic Counter (`atomic.c`) 14

6.2 Performance Evaluation 14

7 CONCLUSIONS **15**

REFERENCES **16**

1 INTRODUCTION AND GOAL

1.1 Introduction

Modern symmetric multiprocessing (SMP) computer architectures allow multiple execution threads to execute concurrently across distinct physical CPU cores. When threads within a single process share the same virtual address space, unrestricted concurrent modifications to mutable shared memory inevitably cause data races.

At the hardware level, high-level language statements such as `counter++` do not translate into single indivisible machine operations. Instead, the instruction set architecture (ISA) decomposes the increment into a read-modify-write sequence: loading the operand from memory into a CPU register, incrementing the register value inside the Arithmetic Logic Unit (ALU), and storing the modified value back to main memory. If the operating system's preemptive scheduler interrupts a thread between these instructions, or if another CPU core accesses the memory location concurrently, intermediate states overlap and arithmetic updates are lost.

To resolve these race conditions and guarantee determinism, modern operating systems and thread runtime environments provide fundamental synchronization primitives:

- **Mutexes (`pthread_mutex_t`):** Mutual exclusion locks that permit at most one thread to enter a designated critical section at any given time.
- **Semaphores (`sem_t`):** Non-negative integer counters governed by atomic decrement (`sem_wait`) and increment (`sem_post`) operations, invented by Edsger Dijkstra to handle both mutual exclusion and signaling.
- **Atomic Operations (`stdatomic.h`):** Lock-free hardware-assisted primitives that execute indivisible memory transactions without operating system context-switching overhead.

1.2 Laboratory Objectives

The primary objectives of this laboratory work are:

a) Demonstrate and inspect real-world data races in multi-threaded C programs under Linux using POSIX threads (`pthreads`).

b) Dissect the underlying CPU instruction cycle of arithmetic increments and explain the mechanics of lost updates.

c) Eliminate data races using POSIX mutual exclusion locks (`pthread_mutex_t`) and POSIX binary semaphores (`sem_t`).

d) Conduct empirical performance and timing comparisons across unsynchronized, mutex-protected, and semaphore-protected executions.

e) Utilize semaphores as unidirectional signaling mechanisms to enforce strict order of execution across asynchronous threads.

f) Utilize counting semaphores to regulate bounded resource capacity (the classic parking lot allocation model).

g) Investigate hardware-level lock-free atomic primitives (`atomic_fetch_add`) as an alternative to software locking.

2 PART 1: OBSERVING THE DATA RACE

2.1 Program Implementation (`race.c`)

To expose a concurrent race condition, a C program was developed where two concurrent POSIX threads increment a shared global 64-bit integer (`counter`) 1,000,000 times each:

```c
/* race.c - two threads add to the same counter, with no protection.
 * Build: gcc -Wall -pthread -o race race.c
 */
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0; /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++; /* read, add 1, write back: three steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

2.2 Execution and Trace Output

The program was compiled without optimization flags (`-O0`) to ensure that the compiler preserves direct memory accesses on each iteration rather than optimizing the loop into CPU registers. The program was executed multiple consecutive times:

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1294812 (expected 2000000)
$ ./race
counter = 1381204 (expected 2000000)
$ ./race
counter = 1178490 (expected 2000000)
```

Figure 2.1: Figure 2.1 – Non-deterministic execution outputs of unsynchronized race.c

2.3 Answers to Required Observation Questions

a) **Write down the three steps the CPU does to execute `counter++`:**

In assembly language on an x86_64 architecture, the high-level statement `counter++` translates into three distinct, non-atomic instructions:

- **1. Memory Load:** The CPU reads the 64-bit quadword from the virtual memory address of `counter` into a general-purpose processor register:

```
1        movq counter(%rip), %rax
```

- **2. Arithmetic Increment:** The ALU increments the register value by 1:

```
1        addq $1, %rax
```

- **3. Memory Store:** The CPU writes the updated value from register `%rax` back to the memory location of `counter`:

```
1        movq %rax, counter(%rip)
```

b) **Explain in words how one addition is lost when two threads do this at the same time:**

Suppose the current value of `counter` stored in RAM is 100:

1) Thread A executes step 1 (**Load**), reading 100 into its local register $\texttt{\%rax}_A$.
2) Before Thread A can perform step 3, a hardware timer interrupt triggers a context switch, or Thread B concurrently executes on another CPU core.
3) Thread B executes step 1 (**Load**), reading the exact same value 100 into its local register $\texttt{\%rax}_B$.
4) Thread A resumes, executes step 2 (**Add**: $\%\text{rax}_A = 101$), and executes step 3 (**Store**), writing 101 to `counter`.
5) Thread B now executes step 2 (**Add**: $\%\text{rax}_B = 101$), and executes step 3 (**Store**), writing 101 over `counter`.

Although two separate additions were performed by the CPU cores, `counter` only transitioned from 100 to 101. Thread B completely overwrote Thread A's modification because both threads operated on stale data, resulting in a permanent lost update.

c) **Why does the number change each time `./race` is executed?**

The Linux Completely Fair Scheduler (CFS) dynamically schedules threads across available CPU cores based on hardware timer ticks, CPU runqueue load, and preemption heuristics. Furthermore, hardware variables such as dynamic frequency scaling, multi-core cache coherency invalidations (MESI protocol), and branch prediction jitter introduce minute timing variations. Because instruction interleavings between the two physical cores are non-deterministic, the exact collision frequency of overlapping load-store windows fluctuates on every execution.

3 PART 2: SYNCHRONIZATION VIA MUTEX AND SEMAPHORE

3.1 Implementation of `mutex.c` and `sem.c`

To prevent concurrent interference, mutual exclusion was enforced using two distinct mechanisms:

a) **POSIX Mutex (`mutex.c`):** Initialized statically with `PTHREAD_MUTEX_INITIALIZER`. The critical section `counter++` is bounded by `pthread_mutex_lock()` and `pthread_mutex_unlock()`.

b) **POSIX Semaphore (`sem.c`):** Initialized dynamically via `sem_init(&s, 0, 1)`. A start value of 1 configures it as a binary semaphore.

```c
/* mutex.c - the same counter, protected by a mutex */
#include <stdio.h>
#include <pthread.h>
#define N 1000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);   /* enter critical section */
        counter++;
        pthread_mutex_unlock(&lock); /* exit critical section */
    }
    return NULL;
}
```

```c
/* sem.c - the same counter, protected by a binary semaphore */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define N 1000000

long counter = 0;
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s); /* decrement s; block if s == 0 */
        counter++;
        sem_post(&s); /* increment s; wake waiter */
    }
    return NULL;
}
```

3.2 Execution and Performance Benchmarking

Both programs were compiled and profiled using the GNU `time` command:

```
$ gcc -Wall -pthread -o mutex mutex.c
$ gcc -Wall -pthread -o sem sem.c

$ time ./race
counter = 1314050 (expected 2000000)
real    0m0.003s
```

```
user    0m0.005s
sys     0m0.000s

$ time ./mutex
counter = 2000000 (expected 2000000)
real    0m0.068s
user    0m0.092s
sys     0m0.012s

$ time ./sem
counter = 2000000 (expected 2000000)
real    0m0.084s
user    0m0.108s
sys     0m0.016s
```

Table 3.1: Table 3.1 – Execution Time and Correctness Comparison

Implementation	Final Counter	Correctness	Execution Time (Real)
Unsynchronized (`race.c`)	≈ 1,300,000	Incorrect (Race Condition)	0.003s
POSIX Mutex (`mutex.c`)	2,000,000	Correct	0.068s
POSIX Semaphore (`sem.c`)	2,000,000	Correct	0.084s

3.3 Answers to Required Observation Questions

a) **Why is the counter always 2,000,000 with the mutex?**

A mutex guarantees absolute **mutual exclusion**. When Thread A executes `pthread_mutex_lock(&lock)`, it acquires ownership of the lock. If Thread B attempts to call `pthread_mutex_lock(&lock)` while Thread A is inside the critical section, Thread B is immediately blocked. As a result, the three underlying CPU steps (load, add, store) are executed sequentially in complete isolation without concurrent interruption, ensuring all 2,000,000 increments register in memory.

b) **In `sem.c`, change the start value to 2 (`sem_init(&s, 0, 2)`). What happened? Why?**

When recompiled and executed with an initial value of 2:

```
    $ ./sem
    counter = 1482110 (expected 2000000)
```

A semaphore value represents the number of available permits. When initialized to 2, two threads are permitted to simultaneously pass `sem_wait(&s)` without blocking. Because both threads enter the critical section concurrently, the mutual exclusion invariant is violated, restoring the race condition and resulting in lost updates.

c) **Why is the locked version slower than `race`?**

The locked version incurs significant runtime latency due to three main factors:

- **Serialization of Work:** Unsynchronized threads execute in true parallel fashion on multiple CPU cores. Synchronization forces thread execution to be serialized sequentially.

- **Atomic Instruction Overhead:** Mutex acquisition in user-space uses atomic instructions (such as `lock cmpxchg` on x86_64) paired with memory barriers. These enforce CPU bus locks and stall hardware execution pipelines to synchronize L1/L2 caches.
- **Kernel Syscall / Futex Transitions:** When high contention occurs, threads that fail to acquire the lock cannot spin indefinitely; they invoke the `sys_futex` system call to transition into kernel sleep, incurring substantial context switch and trap overheads.

4 PART 3: A SEMAPHORE AS A SIGNAL

4.1 Program Implementation (`order.c`)

Semaphores are not merely locks; when initialized to 0, they serve as powerful unidirectional synchronization signals. In `order.c`, Thread B is scheduled first and must wait until Thread A finishes generating a message:

```c
/* order.c - thread B must wait until thread A writes the message */
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready; /* starts at 0: nothing ready yet */

void *writer(void *arg) {
    sleep(1); /* simulate processing latency */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready); /* 0 -> 1: signal thread B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready); /* block while value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL); /* B launched first */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

4.2 Execution Trace

```
$ gcc -Wall -pthread -o order order.c
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"
```

4.3 Answers to Required Observation Questions

a) **In `order.c`, change `sem_init(&ready, 0, 0)` to `sem_init(&ready, 0, 1)`. What happens? Why?**

When initialized to 1, the execution order breaks:

```
$ ./order
B: waiting for the message...
B: I read ""
A: message written
```

Because the semaphore counter starts at 1, Thread B's call to `sem_wait(&ready)` immediately decrements the counter from 1 to 0 without blocking. Thread B proceeds to read the `message` buffer before Thread A has finished sleeping and writing to it, resulting in reading an uninitialized empty string.

b) **In your own words: what does a semaphore that starts at 0 mean?**

A semaphore initialized to 0 represents a **closed gate / event notification signal**. No thread can pass through the gate via `sem_wait()` because no resource permits are initially available. It enforces a strict **happens-before** synchronization relationship: the waiting thread is guaranteed to remain dormant until a notifying thread explicitly completes an operation and opens the gate by posting to the semaphore (`sem_post()`).

5 PART 4: COUNTING SEMAPHORES AND RESOURCE LIMITS

5.1 Program Implementation (`parking.c`)

Counting semaphores are designed to manage multi-unit resource pools. In `parking.c`, a parking lot with 3 spaces is shared among 6 competing car threads:

```c
/* parking.c - 6 cars, 3 places: counting semaphore */
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places; /* tracks available free spaces */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0; /* tracks current occupancy for display */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places); /* claim a free space */

    pthread_mutex_lock(&lock);
    inside++;
    printf("car␣%ld␣enters␣(inside:␣%d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1); /* parked duration */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car␣%ld␣leaves␣(inside:␣%d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sem_post(&places); /* relinquish space */
    return NULL;
}
```

5.2 Execution Trace

```
$ gcc -Wall -pthread -o parking parking.c
$ ./parking
car 1 enters (inside: 1)
car 2 enters (inside: 2)
car 3 enters (inside: 3)
car 1 leaves (inside: 2)
car 4 enters (inside: 3)
car 2 leaves (inside: 2)
car 5 enters (inside: 3)
car 3 leaves (inside: 2)
car 6 enters (inside: 3)
car 4 leaves (inside: 2)
car 5 leaves (inside: 1)
car 6 leaves (inside: 0)
```

5.3 Answers to Required Observation Questions

a) **What is the largest value of `inside` you ever see? Why?**

The largest value of `inside` observed is strictly **3**. The semaphore `places` is initialized to `PLACES = 3`. Each car entering the parking lot calls `sem_wait(&places)`. After three cars enter, the internal counter reaches 0. Any subsequent car thread calling `sem_wait` is immediately blocked by the kernel until an exiting car calls `sem_post(&places)`. Thus, the counting semaphore enforces an upper bound on concurrent occupancy.

b) **If you change `PLACES` to 1, what does the semaphore become?**

When initialized to 1, the counting semaphore degenerates into a **binary semaphore**, which is functionally equivalent to a **mutual exclusion lock (mutex)**. It restricts occupancy to exactly one car at any given moment.

6 EXTENSION: LOCK-FREE HARDWARE ATOMICS

6.1 Lock-Free Atomic Counter (`atomic.c`)

To explore modern hardware-assisted synchronization, an alternative implementation was analyzed utilizing C11 standard atomics (`stdatomic.h`):

```c
/* atomic.c - lockless counter using atomic instructions */
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>

#define N 1000000

atomic_long counter = 0;

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        atomic_fetch_add(&counter, 1); /* atomic hardware instruction */
    return NULL;
}
```

6.2 Performance Evaluation

```
$ gcc -Wall -pthread -o atomic atomic.c
$ time ./atomic
counter = 2000000 (expected 2000000)
real    0m0.018s
user    0m0.034s
sys     0m0.000s
```

At the machine level, `atomic_fetch_add` compiles directly into a single x86_64 instruction prefixed with the bus lock signal:

```
lock addq $1, counter(%rip)
```

Because this instruction is handled entirely within the CPU's hardware memory bus cache-coherency controller, it completely eliminates OS-level context switches, thread sleep states, and futex traps. As demonstrated in Table 6.1, it delivers the exact 2,000,000 result while executing nearly 4× faster than a standard mutex.

Table 6.1: Table 6.1 – Overall Synchronization Performance Benchmark

Mechanism	**Correct Result**	**Runtime**	**Kernel Syscalls**
Unsynchronized (`race`)	No (1.3×10^6)	0.003s	0
Hardware Atomic (`atomic`)	Yes (2.0×10^6)	0.018s	0
POSIX Mutex (`mutex`)	Yes (2.0×10^6)	0.068s	Low (futex on contention)
POSIX Semaphore (`sem`)	Yes (2.0×10^6)	0.084s	Low

7 CONCLUSIONS

In this laboratory work, the fundamental causes, symptoms, and solutions of concurrency race conditions under Linux were investigated.

By dissecting the load-add-store execution cycle of processor instructions, the vulnerability of unprotected shared memory operations was demonstrated empirically. The application of POSIX mutexes and binary semaphores successfully eliminated race conditions by establishing serialized critical sections, albeit with measurable execution latency caused by memory barriers and context switches.

Furthermore, this lab explored the versatility of semaphores:

- When initialized to 0, semaphores operate as event notification signals that enforce temporal execution ordering across asynchronous threads.
- When initialized to positive integers $K > 1$, semaphores operate as resource counters that throttle concurrent access to bounded resource pools.

Finally, evaluating C11 atomic primitives demonstrated that hardware-level atomic instructions provide lightweight, lock-free synchronization for primitive data types without incurring the overhead of operating system lock contention.

REFERENCES

[1] Michael Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.

[2] W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, Addison-Wesley, 3rd Edition, 2013.

[3] Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson, 4th Edition, 2014.

[4] Department of Software Engineering and Automation, *Operating Systems Lab 11: The Race and the Semaphore*, FCIM, Technical University of Moldova, 2026.